

I - Environnement de travail

I.a Environnement Quartus Prime et Questa

- Quartus est l'IDE qui vous permettra de développer des solutions pour les FPGA Intel. Vous pourrez y rédiger votre code, synthétiser vos solutions, consulter les rapports de synthèse, et programmer la carte FPGA. Vous pourrez également appeler le logiciel Questa depuis Quartus.
- Questa est le logiciel qui vous permettra de simuler vos implémentations VHDL.

I.b Rendus

Vous consignerez vos notes et observations **directement dans vos sources** :

- S'il vous est demandé de commenter des résultats, **consignez-vos observations dans un commentaire en en-tête** du fichier contenant votre fonction main.
- Commentez également votre code **dans les lignes**, de manière à expliquer vos choix

Il vous est demandé d'enregistrer des **captures d'écran** de vos simulations dans un dossier que vous créerez dans chaque projet.

Il n'est pas demandé de compte-rendu séparé. Les modalités de rendu des TPs seront discutées lors de la première séance.

Vous remettrez votre travail via [Github](#): initialisez le répertoire TP_FPGA comme un repo Git : **un commit et push vers Github à chaque fin de séance**. Lors de la première séance, chaque binôme communiquera à son encadrant le lien vers le repo Github contenant ses sources via le questionnaire suivant, **à remplir au début de la première séance** :

[Constitution des binômes & liens Github](#)

Pour n'inclure à votre repo que les fichiers nécessaires, **utilisez le fichier [.gitignore](#)** disponible sur [junia-learning](#), **à placer à la racine du repo**.

II - Exercices

Les exercices qui suivent proposent un **exemple élémentaire de synthèse d'un circuit combinatoire** (en l'occurrence un circuit additionneur) suivant la **logique modulaire du design sur FPGA** : chaque fonction numérique est décrite dans un fichier et est **réutilisée** pour **construire des fonctions de plus en plus complexes**.

II.a Synthèse d'un additionneur

Créer un nouveau projet « TP1_additionneur » (voir Annexes)

Dans cette série d'exercices, on décrira **un circuit additionneur en VHDL**. A des fins pédagogiques, on s'attachera à utiliser les **opérateurs logiques** plutôt que d'utiliser les additionneurs présents sur le FPGA (on s'interdira donc l'utilisation du symbole « + »).

II.a.1 Synthèse d'un demi-additionneur 1-bit – Description d'un composant

Le bloc réalisé dans cet exercice peut être décrit comme suit :

Bloc demi-additionneur 1b			
	Nom	Taille	Description
Entrées	A	1 bit	Premier opérande
	B	1 bit	Deuxième opérande
Sorties	S	1 bit	Somme A + B
	C	1 bit	Retenue de A + B

Travail demandé :

1. Rappelez les **opérations logiques** du permettant de calculer *S* et *C* en fonction de *A* et *B*.
2. Ajoutez à votre projet un nouveau fichier half_adder.vhd¹, dans lequel vous décrierez dans une **entity** nommée half_adder le fonctionnement du **demi-additionneur**

II.a.2 Synthèse d'un additionneur complet 1-bit – Instanciation d'un composant en VHDL

Dans cet exercice, on **réutilisera** le **demi-additionneur** précédemment développé pour réaliser un **additionneur complet**. Le bloc réalisé peut être décrit comme suit :

Bloc additionneur 1b complet			
	Nom	Taille	Description
Entrées	A	1 bit	Premier opérande
	B	1 bit	Deuxième opérande
	Cin	1 bit	Retenue à l'entrée
Sorties	S	1 bit	Somme A + B + Cin
	Cout	1 bit	Retenue à la sortie

¹ File > New > VHDL File

Travail demandé :

1. Rappelez comment réaliser un additionneur complet **à partir de deux demi-additionneurs**
2. Ajoutez à votre projet un nouveau fichier `full_adder.vhd`, dans lequel vous décrirez dans une *entity* nommée `full_adder` le fonctionnement de l'additionneur complet. Pour ce faire, vous **instancierez** deux demi-additionneurs précédemment décrits.

II.a.3 Additionneur complet 4-bits

Dans cet exercice, on réutilisera l'additionneur complet 1b précédemment développé pour réaliser un additionneur complet 4b.

Le bloc réalisé peut être décrit comme suit :

Bloc additionneur 4b complet			
	Nom	Taille	Description
Entrées	A	4 bits	Premier opérande
	B	4 bits	Deuxième opérande
	Cin	1 bit	Retenue à l'entrée
Sorties	S	4 bits	Somme A + B + Cin
	Cout	1 bit	Retenue à la sortie

Travail demandé :

1. Rappelez comment réaliser un **additionneur à retenue propagée** à partir d'additionneurs complets
2. Ajoutez à votre projet un nouveau fichier `full_adder_4b.vhd`, dans lequel vous décrirez l'additionneur 4b complet. Pour ce faire, **vous utiliserez des instances de l'additionneur complet** précédemment décrit.
3. Ajoutez à votre projet un nouveau fichier de **testbench** `tb_full_adder_4b.vhd`, dans lequel vous décrirez les entrées à appliquer à l'additionneur 4b complet pour tester son bon fonctionnement
4. Synthétisez et simulez votre additionneur 4b complet

II.a.4 Additionneur complet 4-bits – interaction avec les entrées/sorties de la carte

On souhaite implémenter sur la carte Cyclone V GX l'additionneur réalisé, afin d'en démontrer la fonctionnalité en pratique. On utilisera alors :

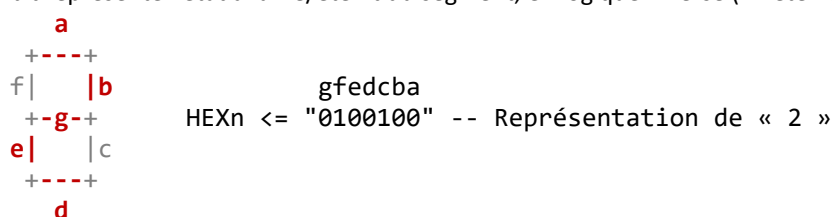
- Les interrupteurs comme entrées (A : SW0-3 ; B : SW4-7 ; Cin : SW9)
- L'afficheur 7-segments HEX3 affichera l'opérande A sous forme hexadécimale
- L'afficheur 7-segments HEX2 affichera l'opérande B sous forme hexadécimale
- L'afficheur 7-segments HEX0 affichera le résultat A+B+Cin sous forme hexadécimale

D'abord, vous devez décrire comment votre module s'interface avec les entrées/sorties de la carte de développement. On appellera communément cette description le **Top Level**, puisqu'il se trouvera au sommet de la hiérarchie de votre solution.

Les entrées/sorties de cette *entity* décrivent les interfaces matérielles de la carte avec lesquelles vos modules s'interfaceront. Vous pouvez lire l'intégralité de ces mots-clés dans le fichier `baseline_c5gx.v`, ajouté automatiquement à vos projets lorsque vous les créez. Ici, vous utiliserez les mots-clés réservés LEDR (sorties vers les 10 LEDs rouges), LEDG (sorties vers les 8 LEDs vertes), et SW (entrées vers les 10 switches).

Vous devrez aussi décrire un bloc combinatoire permettant de transcoder la représentation binaire d'un nombre représenté sur 4 bits (0,1,2,...,9, A,...,E,F) en sa représentation sur afficheur 7 segments².

² Les afficheurs 7 segments de la carte Cyclone V GX utilisent des entrées HEXn sur 7 bits (1 bit par segment) ; les segments sont notés a,b,c,d,e,f,g, avec a le bit de poids faible du mot HEXn et g le bit de poids fort. Chaque bit représente l'état allumé/éteint du segment, en logique inverse (1 : éteint, 0 : allumé)



Travail demandé

1. Créer une entity dans un nouveau fichier *transcodeur_7seg.vhd* permettant de transcoder un nombre représenté sur 4 bits en sa représentation sur afficheur 7 segments, définie ainsi :

Transcodeur 7 segments			
	Nom	Taille	Description
Entrées	BIN	4 bits	Valeur d'entrée encodée en binaire
Sorties	SEG	7 bits	Valeur mise en forme pour les afficheurs 7 segments

2. Ajoutez à votre projet un fichier toplevel.vhd
3. Décrivez l'entity toplevel, dont l'instruction port est :

```
entity toplevel is
    port (
        HEX3 : out std_logic_vector(6 downto 0);
        HEX2 : out std_logic_vector(6 downto 0);
        HEX0 : out std_logic_vector(6 downto 0);
        SW    : in  std_logic_vector(9 downto 0)
    );
end entity;
```

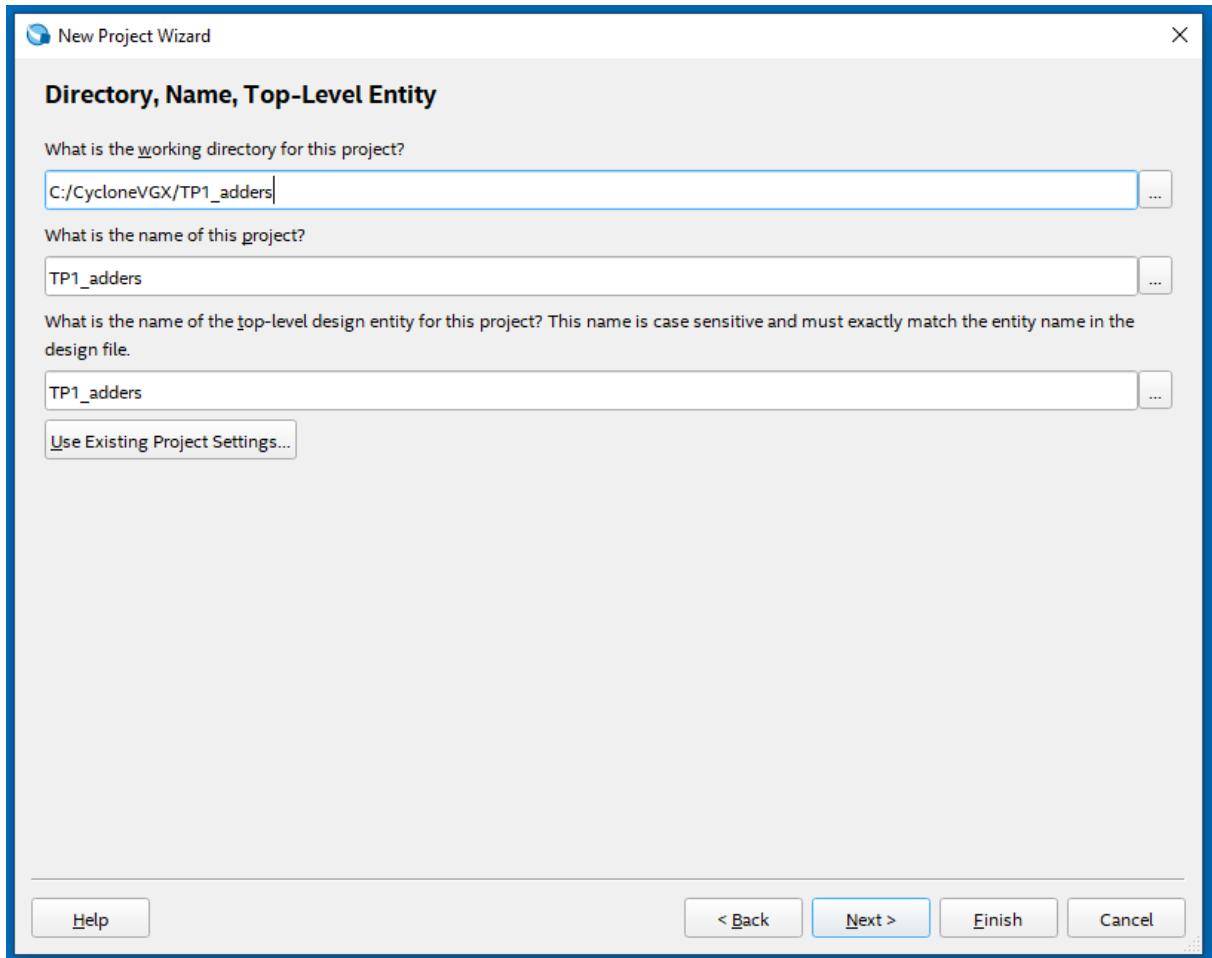
4. Instancier dans l'architecture de toplevel votre additionneur et trois transcodeurs, en connectant adéquatement ses entrées/sorties à HEX3, HEX2, HEX0, et SW.
5. Implémentez votre additionneur sur la carte en utilisant le bouton program device³ et vérifiez qu'il se comporte comme attendu. Félicitations, vous avez implémenté votre première fonction combinatoire sur FPGA.

³ Voir annexe en fin de document

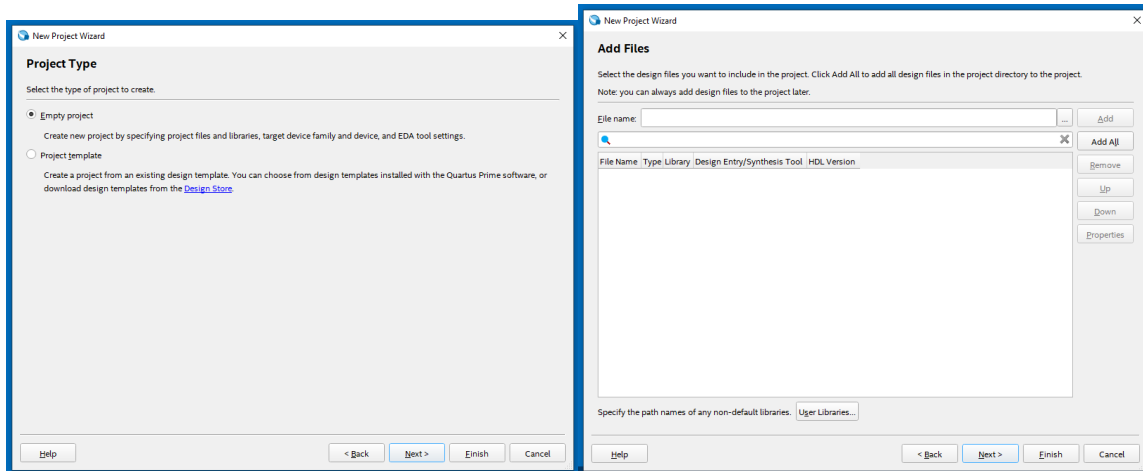
Annexes

Créer un nouveau projet sous Quartus Prime :

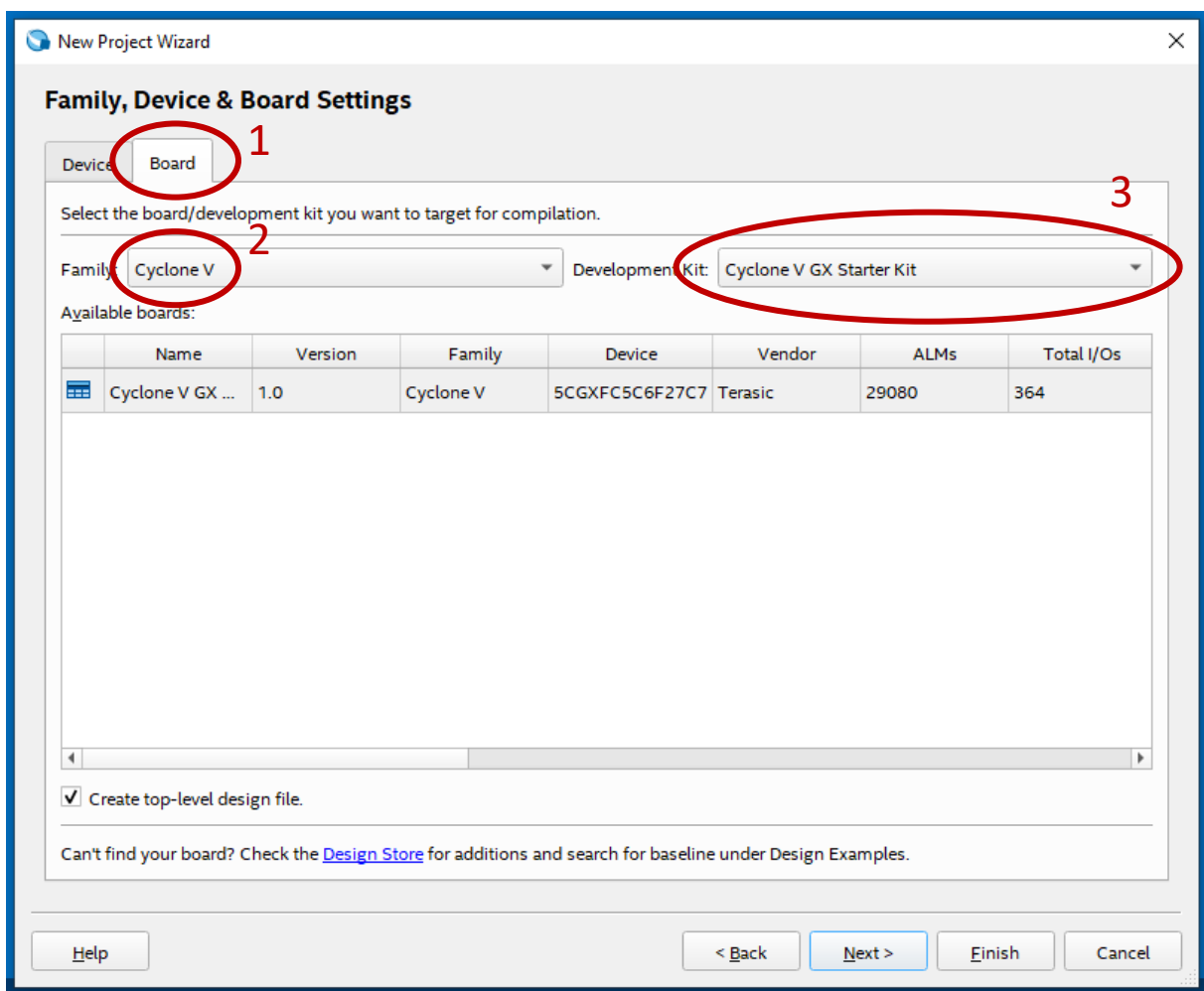
Allez dans File > New Project Wizard... et nommez adéquatement votre nouveau projet.
Cliquez sur Next.



Poursuivez avec les fenêtres «**Project Type** » et «**Add Files** » sans rien modifier.
Cliquez sur Next.



Dans la fenêtre **Family, Device & Board**, sélectionnez Board, puis la famille Cyclone V, puis le kit Cyclone V GX Starter Kit, qui correspond à la carte de développement qui sera utilisée par la suite. Cliquez sur Next.



Enfin, dans **EDA Tool Settings**, renseignez les champs Simulation :

Tool : Questa Intel FPGA et Format : VHDL

Cliquez sur Next, puis terminez.

New Project Wizard

EDA Tool Settings

Specify the other EDA tools used with the Quartus Prime software to develop your project.

EDA tools:

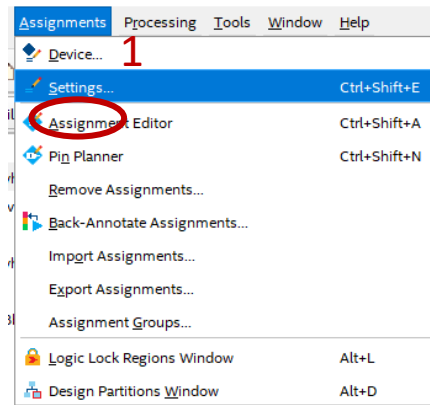
Tool Type	Tool Name	Format(s)	Run Tool Automatically
Design Entry/Synthesis	<None>	<None>	☐ Run this tool automatically to synthesize the current design
Simulation	Questa Intel FPGA	VHDL	☐ Run gate-level simulation automatically after compilation
Board-Level	Timing	<None>	
	Symbol	<None>	
	Signal Integrity	<None>	
	Boundary Scan	<None>	

Help < Back Next > Finish Cancel

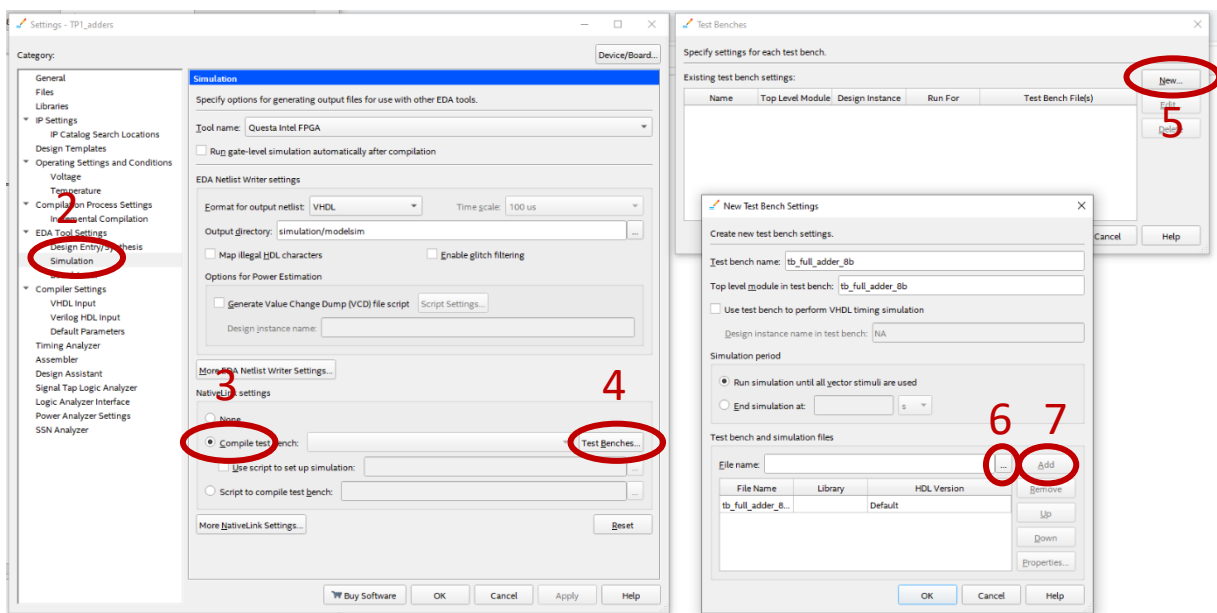
Simuler un testbench

Lorsque vous avez créé un testbench, suivez la procédure suivante pour le simuler :

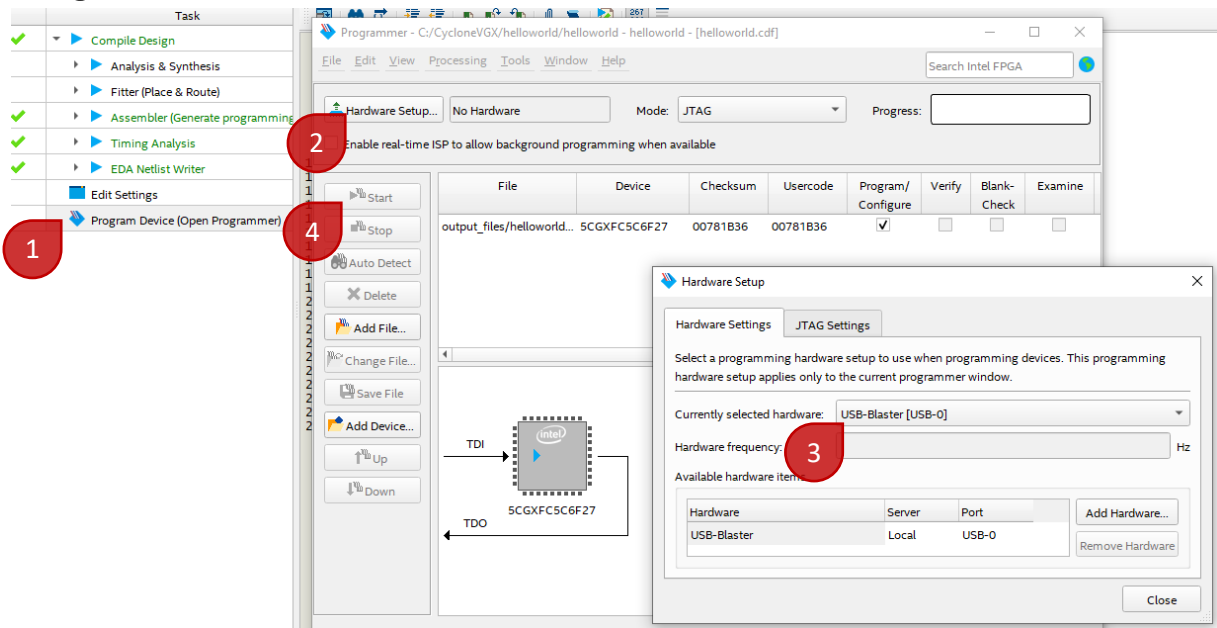
6. Allez dans Assignments > Settings ...
7. Cliquez sur Simulation
8. Cliquez sur Compile testbench
9. Ajoutez le fichier de testbench en cliquant sur Test Benches
10. Cliquez sur New ...
11. Nommez le testbench à ajouter, puis ajoutez le fichier .vhd correspondant en cliquant sur « ... »
- Vous pouvez également à ce point renseigner la durée de la simulation en renseignant le champ « End simulation at » (recommandé)
12. Validez l'ajout du fichier vhd en cliquant sur « Add », puis validez toutes les fenêtres.



Une fois cette procédure suivie, vous pourrez lancer une simulation de votre module top-level en allant dans Tools > Run Simulation Tool > ...



Programmation de la carte



Si vous ne trouvez pas l'USB Blaster, référez-vous au document « Annexe Quartus Prime » sur junia-learning.